

AIML

NLP and Generative AI

1.1 Introduction to Text Mining: Introduction, Algorithms for Text Mining, Future Directions

Text mining, also referred to as text data mining or text analytics, is the process of **extracting valuable insights and knowledge from unstructured text data**. It achieves this by using natural language processing (NLP), machine learning, and statistics to analyse and interpret large amounts of textual information. The text mining market has grown significantly in recent years and is expected to continue to do so. This growth is primarily due to increased competition in the business market, with many organisations seeking value-added solutions to compete.

Objectives of Text Mining

- **Information Retrieval:** Text mining helps to extract relevant information from a vast amount of text.
- **Knowledge Discovery:** It helps to identify patterns, trends, and hidden relationships in textual data.
- **Sentiment Analysis:** Text mining is used to analyse opinions, sentiments, and emotions expressed in text.
- **Document Categorisation:** It classifies documents into predefined categories or topics.

Algorithms for Text Mining

- **Information Extraction:** Information extraction involves automatically extracting structured data like entities, relationships, and attributes from unstructured sources. Subtasks include pre-processing text (tokenization, sentence splitting, morphological analysis), finding and classifying concepts, connecting the concepts, unifying data into a standard form, removing duplicate data, and enriching the knowledge base.
- **Unsupervised Learning:** Unsupervised learning algorithms analyse and cluster unlabelled datasets to discover hidden patterns or data groupings without human intervention.
- **Text Summarisation:** This approach involves algorithms or programs that reduce text size and create a summary of the text data. Extractive summarisation uses simple algorithms, while abstractive summarisation exchanges sentences for shorter sentences with the same semantic meaning.

Future Trends in Text Mining

- **Deep Learning:** The integration of neural networks is expected to improve text representation and understanding.
- **Multimodal Text Analysis:** Combining text with other data modalities like images and videos is an emerging trend.
- **Ethical Considerations:** Addressing biases and ensuring responsible use of text mining technologies are increasingly important.

1.2 Information Extraction from Text: Named Entity Recognition, Part-of-Speech (POS) Tagging

Information extraction is the process of finding structured information from unstructured or semi-structured text, which is an important task in text mining that has been studied across multiple research communities. Information extraction enables the automation of tasks like smart content classification, integrated search, and data-driven activities like uncovering hidden relationships.

Named Entity Recognition (NER)

- NER is a natural language processing (NLP) technique that automatically identifies named entities in a text and classifies them into predefined categories.
- Entities can include names of people, organisations, locations, times, quantities, monetary values, and percentages.
- NER can extract key information to understand what a text is about or to collect important information to store in a database.
- For example, a NER model can process the sentence "John has been working at Amazon as a Full Stack Developer" to identify John as a name, Amazon as an organisation, and Full Stack Developer as a job profile.
- NER automatically categorises data from text based on its previous training and isn't affected by large workloads.

Part-of-Speech (POS) Tagging

Part-of-speech (POS) tagging is a fundamental technique in Natural Language Processing (NLP) that identifies and labels words in text according to their grammatical functions. In the context of AI and machine learning for NLP and generative AI, POS tagging serves as an essential preprocessing step that enables more sophisticated language understanding and generation.

How POS Tagging Works

POS tagging assigns labels such as noun, verb, adjective, or adverb to each word in a sentence based on both its definition and context. For example, in the sentence "I need to book a flight," the word "book" is identified as a verb rather than a noun because of its surrounding context.

Implementation in Machine Learning

Modern POS tagging systems use various machine learning approaches:

1. **Statistical Models:** Systems like Hidden Markov Models (HMMs) use probability distributions to determine the most likely tag sequence for a given sentence.
2. **Neural Networks:** Deep learning architectures, particularly recurrent neural networks (RNNs) and transformers, have significantly improved tagging accuracy by capturing complex contextual relationships between words.

1.2 Relation Extraction, Unsupervised Information Extraction

Relation Extraction

- Relation Extraction involves identifying and categorising relationships between entities mentioned in text, going beyond Named Entity Recognition (NER) to understand the connections between entities.
- It helps uncover meaningful associations between different pieces of information and is useful in knowledge graph construction and building structured databases.
- Challenges include ambiguity in defining relationships and handling complex and nuanced connections.
- For example, in the sentence "Bill Gates co-founded Microsoft with Paul Allen," relation extraction would identify the "co-founder" relationship between "Bill Gates" and "Microsoft" and between "Paul Allen" and "Microsoft".
- Relations are one of the most important aspects for building knowledge graphs, which is significant to various NLP applications like summarisation, Q&A, search and sentiment analysis.

Difference between Named Entity Recognition (NER) and Relation Extraction in the sample sentence and explaining the key differences between these processes.

NER Analysis

For the sentence: "John has been working at Amazon as a Full Stack Developer"

NER would identify:

- "John" → PERSON
- "Amazon" → ORGANIZATION
- "Full Stack Developer" → JOB_TITLE

NER focuses solely on identifying and categorizing specific entities in the text without establishing connections between them.

Relation Extraction Analysis

For the same sentence, Relation Extraction would:

1. First identify the entities (similar to NER)
2. Then establish the relationships between them:
 - Relation: WORKS_FOR
 - Subject: John (PERSON)
 - Object: Amazon (ORGANIZATION)
 - Relation: HAS_ROLE
 - Subject: John (PERSON)
 - Object: Full Stack Developer (JOB_TITLE)

Key Differences

1. **Purpose:**
 - NER only identifies and classifies standalone entities
 - Relation Extraction captures the semantic connections between those entities
2. **Output Structure:**
 - NER output: A flat list of entities with their types
 - Relation Extraction output: A structured representation of connections (typically as triples: subject-relation-object)
3. **Complexity:**
 - NER is generally simpler, focusing on identification and classification
 - Relation Extraction requires more complex understanding of language semantics to determine how entities interact
4. **Applications:**
 - NER is useful for entity-centric tasks like content categorization
 - Relation Extraction enables knowledge graph construction and question answering

In our example, NER tells us "who" and "what" exists in the text, while Relation Extraction tells us that John is employed by Amazon and that his role there is as a Full Stack Developer.

Unsupervised Information Extraction

- Unsupervised Information Extraction refers to methods that don't rely on labelled training data, aiming instead to discover patterns and relationships autonomously.
- Techniques include clustering, topic modelling, and pattern learning.
- Advantages of unsupervised methods are that they don't require manually labelled datasets and can discover unexpected patterns and relationships.
- Challenges include potentially less accurate results compared to supervised methods and difficulty in handling noise and ambiguity.
- For example, ReVerb finds United to the left and Chicago to the right of 'has a hub in', and skips over which to find Chicago to the left of 'is the headquarters of'.
- Effective information extraction often involves integrating multiple methods, such as using NER to identify entities, relation extraction to establish connections, and unsupervised methods for discovering additional insights. Combined approaches are used in domains like biomedical research, financial analysis, and social media monitoring to extract structured information from unstructured text data.

1.3 Text Representation: Word2Vec, GloVe, and FastText Embeddings in NLP Applications

tokenization

Text representation involves converting raw text into a suitable numerical form for use in machine learning models. The raw text is preprocessed through tokenization, stop word removal, punctuation removal, and stemming to clean the noise.

Tokenization is a crucial initial step in text preprocessing, which involves breaking down text into smaller units called tokens. Tokens can be characters, words, parts of words, punctuation, sentences or regular expressions. Tokenization is essential for word-level analysis and facilitates feature extraction. For example, the sentence "Natural language processing is fascinating," becomes ["Natural", "language", "processing", "is", "fascinating"] after tokenization.

Word2Vec

- Word2Vec uses the weights of a neural network's hidden layer as features of words.
- It can predict a context or a word based on nearby words in the corpus using continuous bag-of-words or skip-gram models, combined with a 1-1-1 neural network.
- Word2Vec provides better semantic/syntactic relationships of words through vectors.

GloVe and FastText Embeddings

The provided documents do not contain information about GloVe and FastText embeddings.

1.3 stemming, stop words, NER, N-gram modelling

Text representation involves transforming raw text data into a format suitable for analysis, and several techniques are used to achieve this. These techniques, including stemming, stop word removal, Named Entity Recognition (NER), and N-gram modelling, contribute to the effectiveness of natural language processing applications by capturing semantic meaning, reducing noise, and revealing patterns within textual information.

- **Stemming:** Stemming is the process of reducing words to their base or root form by removing suffixes. For example, the words "running" would be stemmed to "run". Stemming reduces words to a common base, capturing the core meaning and helping group variations of a word together. Stemming assists in retrieving relevant documents, eliminating the out-of-vocabulary (OOV) problem and enhancing the accuracy of ML/DL models.
- **Stop Words:** Stop words are common words such as "the", "is", and "and", that are often removed during text processing because they carry little semantic meaning. Removing stop words eliminates noise and reduces dimensionality in text data, focusing analysis on content-carrying words. In the sentence "The quick brown fox jumps over the lazy dog," stop words would be removed, leaving important words like "quick," "brown," "fox," "jumps," "lazy," and "dog".
- **Named Entity Recognition (NER):** NER identifies and classifies entities, such as names of people, organisations, locations, dates, etc., in a text. For example, in the sentence "Apple Inc. was founded by Steve Jobs in Cupertino," NER would identify "Apple Inc." as an organisation, "Steve Jobs" as a person, and "Cupertino" as a location. NER enhances information extraction by identifying specific entities and supports tasks like information retrieval and sentiment analysis.
- **N-gram Modelling:** N-gram modelling involves breaking down a sequence of words into contiguous chunks of N words, known as N-grams. In the sentence "I love natural language processing," a bigram (2-gram) model would generate the pairs: ["I love", "love natural", "natural language", "language processing"]. N-gram modelling captures local word patterns and context in a text and is used in language modelling, machine translation, and text generation. For N-1 words, N-gram modelling predicts the most frequently occurring words that can follow the sequences.

2.1. Text Clustering: Feature Selection and Transformation Methods, distance based Clustering Algorithms, Word and Phrase based Clustering, Probabilistic document Clustering

1. Feature Selection and Transformation Methods

- **TF-IDF (Term Frequency-Inverse Document Frequency):** A statistic showing a word's importance in a document relative to a collection of documents. It emphasizes rare words and reduces the impact of common words, which helps in feature representation. The term frequency for each word is normalized by the inverse document frequency (IDF). The IDF normalization reduces the weight of terms that occur more frequently in

the collection, ensuring that the matching of documents is more influenced by more discriminative words which have relatively low frequencies in the collection.

- **Word Embeddings:** Represent words as dense vectors in a continuous vector space to capture semantic relationships and improve feature representation.
- **Dimensionality Reduction:** Techniques like PCA reduce the number of features while preserving essential information, reducing computational complexity and focusing on key features. Latent Semantic Indexing (LSI), Probabilistic Latent Semantic Analysis (PLSA) and Non-negative Matrix Factorization (NMF) are available to improve the quality of document representation and make it more amenable to clustering. In these dimension reduction techniques, the correlations among the words in the lexicon are leveraged in order to create features, which correspond to the concepts or principal components in the data.

2. Distance-based Clustering Algorithms

- Distance-based clustering uses a similarity function to measure the closeness between text objects. The most well-known similarity function used in text is the cosine similarity function.
- **K-Means Clustering:** A partitioning method that divides data into k clusters by minimizing the variance within each cluster. It is efficient, simple, and effective. However, it can be sensitive to the choice of seeds. A vanilla k-means algorithm will select k documents from the original data as the initial seeds. The number of distinct words in these k documents will typically be a very small subset of the entire lexicon, which can cause incoherence in early iterations.
- **Hierarchical Clustering:** Constructs a tree of clusters (dendrograms) by iteratively merging (agglomerative) or splitting (divisive) clusters. It captures hierarchical relationships within the data. Agglomerative clustering starts with each document in its own cluster and creates the hierarchy by merging pairs of clusters. Divisive clustering starts with all documents in the same cluster and creates the hierarchy by splitting clusters recursively.
- **DBSCAN (Density-Based Spatial Clustering of Applications with Noise):** Clusters dense regions of points, separating sparse regions as noise. It is effective for irregularly shaped clusters and noise handling.
- **Single Linkage Clustering:** The similarity between two groups of documents is the greatest similarity between any pair of documents from these two groups. In single link clustering the two groups which are such that their closest pair of documents have the highest similarity compared to any other pair of groups are merged. A main advantage of single linkage clustering is that it is extremely efficient to implement in practice.
- **Complete Linkage Clustering:** merges two clusters with the smallest maximum distance. Yields "compact" clusters and merges only if there are many high-similarity pairs.
- **Average Linkage Clustering:** merges two clusters with the smallest average distance and is less sensitive to outliers than single or complete link.

3. Word and Phrase-Based Clustering

- These methods view the problem in a dual way, finding important clusters of words that may be utilized for finding clusters of documents.
- **Word Embedding Clustering:** Clusters based on the embedding vectors of words, grouping words with similar contexts to capture semantic relationships and context similarities.
- **Phrase-Based Clustering:** Clusters phrases or multi-word expressions based on semantic or syntactic similarities. This method treats a document as a string as opposed to a bag of words and retains ordering information for the clustering process.

4. Probabilistic Document Clustering

- **Topic Modeling:** Creates a probabilistic generative model for text documents. It represents a corpus as a function of hidden random variables, whose parameters are estimated using a document collection. Topic models can be considered as the process of clustering with a generative probabilistic model.
- **Gaussian Mixture Model (GMM):** Represents a dataset as a mixture of several Gaussian distributions. Useful for probabilistic clustering and accommodating data with multiple underlying distributions.
- **Probabilistic text clustering:** can be considered an unsupervised version of the Naive Bayes classification method.

2.2. Text Classification: Feature Selection, Decision tree Classifiers, Rule-based Classifiers, Probabilistic based Classifiers, Proximity based Classifiers

1. Feature Selection Feature selection is a critical initial step in text classification, aiming to identify the most relevant features for the classification process. Due to the high dimensionality and presence of irrelevant (noisy) features in text data, effective feature selection is essential.

- **Importance:**
 - Improves classification accuracy by removing irrelevant features that can harm model performance.
 - Enhances computational efficiency by reducing the number of features the model needs to process.
- **Methods:**
 - **Stop-word Removal:** Eliminates common words (e.g., "the") that don't contribute to distinguishing between classes.
 - **Stemming:** Reduces words to their root form, consolidating variations of a word (e.g., "running," "runs," and "ran" become "run").
 - **Supervised Feature Selection:** Methods that use class labels to select features highly skewed toward a particular class. Examples include the Gini index and entropy.

- **Document Representation:**
 - **Bag of Words:** Represents a document as a set of words and their frequencies, disregarding word order.
 - **String Representation:** Represents text directly as a sequence of words, preserving word order.

2. Decision Tree Classifiers Decision tree classifiers use a hierarchical structure of decisions based on feature values to classify instances.

- **Structure:**
 - **Nodes:** Represent decision points based on features.
 - **Edges:** Connect nodes, indicating possible outcomes.
 - **Leaves:** Contain the predicted class labels.
- **Decision Making:**
 - The data space is divided based on conditions (predicates) on the presence or absence of words in the document.
 - The division continues recursively until leaf nodes meet certain conditions, such as a minimum number of records or class purity.
 - A test instance traverses the tree from top to bottom, based on its features, to reach a leaf node, which determines its class.
- **Advantages:**
 - Easy to understand and interpret.
 - Can handle both numerical and categorical data.
- **Disadvantages:**
 - Prone to overfitting, especially with deep trees.
 - Sensitive to small variations in the data.
- **Example:** A node might split documents based on the presence or absence of the word "health". Documents containing "health" go to one branch, while those without go to another. This process repeats at each node, using different terms, until a classification can be made.

3. Rule-Based Classifiers Rule-based classifiers model the data space with a set of rules. Each rule consists of a condition on the feature set (left-hand side) and a class label (right-hand side).

- **Components:**
 - **Rules:** Define conditions based on word patterns that predict a class label.
 - **Rule Set:** The collection of rules forming the classification model.
- **Classification Process:**
 - For a test instance, identify the rules whose conditions it satisfies.
 - Predict the class label based on the class labels of the satisfied rules.
- **Advantages:**
 - High interpretability, making it easy to understand the decision-making process.

- Easy to maintain and modify, allowing for manual addition of rules based on expert knowledge.
- **Example:** A rule might state, "IF the document contains 'sports' AND 'game,' THEN classify as 'sports'".
- **Relation to Decision Trees:** Decision trees can be transformed into rule-based classifiers, where each path in the tree becomes a rule. Unlike decision trees, rule-based classifiers allow overlaps in the decision space, providing more flexibility.

4. Probabilistic-Based Classifiers Probabilistic classifiers use probability to relate feature variables to a target (class) variable.

- **Types:**
 - **Bayes Classifier:** Uses Bayes' rule to model the probability of each class given a set of features. It often assumes class-conditional feature independence (naive Bayes) to simplify the modeling.
 - **Naive Bayes Classifier:** A generative classifier that models the distribution of documents in each class, assuming independence between the distributions of different terms.
 - **Bernoulli Multivariate Model:** Treats a document as a set of distinct words, ignoring frequency information.
 - **Multinomial Distribution:** Considers word frequencies in the document.
- **Advantages:**
 - Effective for text classification, especially when combined with other features like URLs.
 - Can be adapted for hierarchical classification, building a classifier at each node to guide the classification process.
- **Example:** A Naive Bayes classifier calculates the probability of a document belonging to a class (e.g., "sports") based on the presence of words like "ball," "team," and "score," assuming these words are independent of each other.

5. Proximity-Based Classifiers Proximity-based classifiers use distance measures to perform classification. The main idea is that documents in the same class are likely to be close to each other based on similarity measures.

- **Methods:**
 - **K-Nearest Neighbors (k-NN):** Classifies a test instance based on the majority class among its k nearest neighbors in the training data. The choice of k typically ranges from 20 to 40.
- **Similarity Measures:**
 - Common measures include the dot product and cosine metric.
- **Advantages:**
 - Simple to implement and understand.

- **Example:** To classify a new document, the k-NN classifier finds the 30 closest documents in the training set. If 22 of those 30 documents are labeled "science," the classifier assigns the new document to the "science" category.

2.3. Text Modelling: Bayesian Networks, Hidden Markovian Models, Markov random Fields, Conditional Random Fields

1. Bayesian Networks

- **Definition:** Bayesian Networks, also known as Bayesian Belief Networks, are **graphical models** that represent **probabilistic relationships** among a set of variables. They use directed acyclic graphs (DAGs) to model dependencies between variables.
- **Text Modelling Application:** In natural language processing (NLP), Bayesian Networks can model relationships between words or phrases, capturing dependencies in the structure of sentences or documents.
- **Advantages:**
 - Handles uncertainty through probabilistic representation.
 - Provides a clear graphical structure for understanding dependencies.
- **Disadvantages:** Limited in handling cyclic dependencies.
- **Conditional Independence:** A node is conditionally independent of its non-descendants given its parents. This is also called the local Markov property.
- **Applications in Text Mining:** Bayesian networks have been widely used in many applications in text mining, such as spam filtering and information retrieval. For example, to identify spam email by making use of a naive Bayes classifier.

2. Hidden Markov Models (HMM)

- **Definition:** Hidden Markov Models are **probabilistic models** with **observable and hidden states**. They assume that the system being modelled is a Markov process with unobservable (hidden) states influencing observed events. HMMs are used for clustering, classification, and outlier detection.
- **Text Modelling Application:** HMMs are effective in sequence labelling tasks, like part-of-speech tagging and speech recognition. They can capture sequential dependencies in text. For domain-specific summarization, Hidden Markov Models (HMM) that capture "story flow"—what topics are discussed in what order in the domain— can be trained.

- **Advantages:**
 - Well-suited for sequential data.
 - Can model hidden states that influence observed outputs.
- **Disadvantages:** Assumes the Markov property, which may not always hold.
- **Formal Definition:** A Hidden Markov Model contains n states. Symbols are generated from the model by a sequence of transitions from one state to the other.
- **Techniques for HMMs:**
 - Training: Use Baum–Welch algorithm to construct a separate HMM model for each of the k classes.
 - Testing: For a given test sequence, determine the fit probability of the sequence to the k different Hidden Markov Models. Report the class for which the corresponding HMM has the highest fit probability to the test sequence.

3. Markov Random Fields (MRF)

- **Definition:** A Markov random field (MRF), also known as an undirected graphical model, has a set of nodes, each of which corresponds to a variable or group of variables, as well as a set of links, each of which connects a pair of nodes. The links are undirected.
- **Text Modelling Application:** MRFs model dependencies between words or documents in a text corpus. They are used in text categorisation and information retrieval.
- **Advantages:**
 - Can model complex dependencies without directionality.
 - Allows for incorporating various text features as evidence.
- **Disadvantages:** Inference can be computationally intensive.
- **Applications in Text Mining:** MRF has been widely used in many text mining tasks, such as text categorisation and information retrieval.

4. Conditional Random Fields (CRF)

- **Definition:** A CRF is an undirected graph whose nodes can be divided into exactly two disjoint sets: the observed variables X and the output variables Y . It is a discriminative probabilistic graphical model.
- **Text Modelling Application:** Applied in tasks such as named entity recognition, part-of-speech tagging, and information extraction, where the goal is to predict structured

outputs given input features.

- **Advantages:**
 - Flexible and effective for structured prediction tasks.
 - Takes into account both input and output variables.
 - Relaxes the independence assumptions required by HMMs.
- **Disadvantages:** Requires labelled training data for both input and output variables.
- **Difference Between CRFs and MEMMs:** The major difference between CRFs and MEMMs is that in CRFs, the label of the current observation can depend not only on previous labels but also on future labels. CRFs are undirected graphical models while both HMMs and MEMMs are directed graphical models.

3.1 Introduction to Generative Models, Overview of Generative Adversarial Networks (GANs) and Variational Autoencoders (VAEs)

Introduction to Generative Models

Generative models represent one of the most fascinating areas in machine learning. Unlike discriminative models that learn boundaries between classes (answering "which category does this belong to?"), generative models learn the underlying distribution of data to answer a more fundamental question: "what does data from this category look like?"

What are Generative Models?

At their core, generative models aim to understand the hidden patterns and structures in data well enough to generate new, similar examples. Imagine having seen thousands of faces and then being able to draw a completely new, realistic face that doesn't belong to any real person. That's what generative models can do, but for complex data types including images, text, audio, and more.

Generative models learn a probability distribution $P(X)$ over the data, or a joint distribution $P(X,Y)$ over both data and labels. This allows them to:

1. Generate new samples that resemble training data
2. Complete partial data (fill in missing parts)
3. Learn useful representations of data in an unsupervised manner
4. Model the underlying process that created the data

Generative Adversarial Networks (GANs)

GANs consist of two neural networks locked in a minimax game:

1. **Generator (G):** Attempts to create data that looks real
2. **Discriminator (D):** Tries to distinguish between real data and the generator's fake data

These networks are trained simultaneously with opposing objectives:

- The generator tries to fool the discriminator by creating increasingly realistic samples
- The discriminator tries to get better at telling real from fake

How GANs Work

The training process resembles a counterfeiter (generator) and detective (discriminator) continuously improving against each other:

1. The generator takes random noise as input and produces synthetic data
2. The discriminator evaluates both real data and the generator's output
3. The generator learns to produce more convincing fakes
4. The discriminator learns to detect more subtle flaws

Challenges with GANs

- **Training instability:** The two networks can fail to converge properly
- **Mode collapse:** The generator produces limited varieties of outputs
- **Evaluation difficulty:** Objectively measuring GAN performance is challenging
- **Need for large datasets:** GANs typically require substantial training data

Variational Autoencoders (VAEs)

VAEs are built on two key concepts:

1. Autoencoders: Neural networks that learn efficient encodings of data
2. Variational inference: A method to approximate complex probability distributions

Unlike traditional autoencoders that map inputs to fixed points in a latent space, VAEs map inputs to probability distributions, creating a continuous, structured latent space.

How VAEs Work

A VAE consists of:

1. **Encoder:** Maps input data to parameters of a probability distribution (typically mean and variance of a Gaussian)
2. **Latent space:** A compressed representation where similar data points cluster together

3. **Sampling:** Drawing samples from the encoded distribution using the reparameterization trick
4. **Decoder:** Reconstructs data from latent space samples

VAEs are trained to minimize two terms:

- **Reconstruction loss:** How well the decoder reconstructs the original input
- **KL divergence:** How closely the encoded distributions match a prior (usually a standard normal distribution)

Properties of VAEs

- **Stable training:** Generally more stable than GANs
- **Meaningful latent space:** The latent space is continuous and organized
- **Probabilistic framework:** Provides uncertainty estimates and allows sampling
- **Principled approach:** Based on solid statistical foundations

Challenges with VAEs

- **Blurry outputs:** Often produce less sharp results than GANs
- **Posterior collapse:** The model might ignore parts of the latent space
- **Balance of loss terms:** Difficult to balance reconstruction quality and distribution matching
- **Limited flexibility:** The choice of tractable distributions can constrain expressiveness

Comparing GANs and VAEs

These two frameworks represent different approaches to generative modeling with complementary strengths:

Aspect	GANs	VAEs
Training approach	Adversarial game	Maximum likelihood estimation
Output quality	Often sharper, more realistic	Typically smoother, sometimes blurrier
Latent space	Less structured	More organized, continuous
Training stability	Can be unstable	Generally more stable

Sampling	Direct generation from noise	Encoding-decoding process
Theoretical foundation	Game theory	Variational inference
Evaluation	Harder to evaluate objectively	Can use likelihood estimates

Applications of Generative Models

- **Image generation:** Creating photorealistic images and artwork
- **Image-to-image translation:** Converting sketches to photos, day to night scenes
- **Text generation:** Creating coherent and contextually appropriate text
- **Music composition:** Generating novel musical pieces in various styles
- **Drug discovery:** Designing molecules with specific properties
- **Data augmentation:** Creating synthetic training examples for other models
- **Anomaly detection:** Identifying unusual patterns by modeling normal data

3.2 Text Generation with GPT (Generative Pre-trained Transformer), Understanding Transformer Architecture, Fine-tuning GPT for Specific Tasks, Applications of Generative AI

Understanding Transformer Architecture

The Transformer architecture represents one of the most significant breakthroughs in natural language processing (NLP). Unlike previous sequence models that relied on recurrent or convolutional mechanisms, Transformers use a novel attention mechanism that allows the model to weigh the importance of different words in relation to each other, regardless of their distance in the sequence.

Key Components of the Transformer Architecture

1. Self-Attention Mechanism

At the heart of the Transformer lies the self-attention mechanism, which allows the model to consider relationships between all words in a sequence simultaneously:

- **Query, Key, Value (QKV):** Each word is transformed into three vectors – a query, a key, and a value.
- **Attention Calculation:** For each position, we compute attention scores by comparing its query with all other positions' keys.

- **Weighted Sum:** The final representation of each position is a weighted sum of all values, with weights determined by the attention scores.

2. Multi-Head Attention

Rather than performing a single attention function, Transformers employ multiple attention heads that operate in parallel:

- Each head has its own set of learned parameters for projecting inputs to queries, keys, and values
- This allows the model to attend to information from different representation subspaces
- The outputs from all heads are concatenated and linearly transformed

The multi-head attention enables the model to simultaneously attend to different aspects of the input, such as syntactic and semantic relationships.

3. Position Encoding

Since the Transformer processes all words in parallel (rather than sequentially like RNNs), it needs an explicit way to understand word order:

- Positional encodings are added to the input embeddings
- These are typically sine and cosine functions of different frequencies
- This gives the model information about the relative or absolute position of tokens

4. Feed-Forward Networks

Each layer in the Transformer includes a position-wise feed-forward network:

- Applied to each position independently and identically
- Consists of two linear transformations with a ReLU activation in between
- Can be viewed as a way to process the attention output further with more non-linearity

5. Layer Normalization and Residual Connections

To facilitate training deep networks:

- Layer normalization stabilizes the activations
- Residual connections (skip connections) help with gradient flow during backpropagation
- Together, they allow Transformers to scale to dozens of layers

Transformer-Based GPT Models

GPT (Generative Pre-trained Transformer) models build upon the Transformer architecture with some key differences:

1. **Unidirectional Context:** While the original Transformer used bidirectional attention (as in BERT), GPT uses a masked self-attention mechanism where each position can only attend to itself and previous positions, making it unidirectional or "causal."
2. **Pre-training and Fine-tuning Approach:** GPT models are first pre-trained on vast amounts of text data through unsupervised learning, then fine-tuned on specific tasks.
3. **Scaling Properties:** GPT models have demonstrated remarkable scaling properties - as the model size and training data increase, capabilities improve in predictable ways.
4. **In-Context Learning:** Larger GPT models show an emergent ability to perform tasks from instructions and examples provided in the prompt, without explicit fine-tuning.

Fine-tuning GPT for Specific Tasks

While pre-trained GPT models already possess broad capabilities, fine-tuning adapts them to specific domains or tasks with relatively small amounts of labeled data.

Fine-tuning Approaches

1. Full Fine-tuning

This involves updating all model parameters:

- The entire model is trained on task-specific data
- Requires significant computational resources for large models
- Produces a specialized model that may lose some general capabilities

2. Parameter-Efficient Fine-tuning

Several techniques modify only a small subset of parameters:

- **Low-Rank Adaptation (LoRA):** Adds trainable low-rank matrices to existing weights
- **Prefix Tuning:** Prepends trainable prefix tokens to the input
- **Prompt Tuning:** Similar to prefix tuning but typically using fewer parameters
- **Adapter Layers:** Inserts small trainable modules between Transformer layers

These approaches typically require only 0.1-1% of the parameters to be updated, drastically reducing computational and storage requirements.

3. Fine-tuning with Reinforcement Learning from Human Feedback (RLHF)

This more complex approach combines supervised fine-tuning with reinforcement learning:

1. **Supervised Fine-tuning:** Initial adaptation to the task
2. **Reward Model Training:** A separate model learns to predict human preferences
3. **Reinforcement Learning Optimization:** The model is optimized using the reward model

RLHF has been crucial for aligning models with human preferences and reducing harmful outputs.

Fine-tuning Process

A typical fine-tuning workflow includes:

1. **Data Preparation:** Collecting and formatting task-specific data
2. **Hyperparameter Selection:** Choosing learning rate, batch size, and training duration
3. **Training Loop:** Updating model parameters based on task-specific objective
4. **Evaluation:** Assessing performance on validation data
5. **Deployment:** Implementing the fine-tuned model in production

Challenges in Fine-tuning

Several challenges arise when fine-tuning large language models:

- **Catastrophic Forgetting:** The model may lose previously acquired knowledge
- **Overfitting:** Especially with small fine-tuning datasets
- **Computational Requirements:** Full fine-tuning of large models requires substantial resources
- **Evaluation Complexity:** Determining when a model is properly fine-tuned can be difficult

Applications of Generative AI

Natural Language Processing

1. **Content Creation:**
 - Writing assistants that help draft emails, reports, and creative content
 - Marketing copy generation for ads, product descriptions, and social media posts
 - Automated blog post and article creation
2. **Language Translation and Localization:**
 - Neural machine translation systems with increased fluency
 - Context-aware cultural adaptation of content

Education and Knowledge Access

1. Personalized Learning:

- Adaptive tutoring systems that respond to student questions
- Content generation at different complexity levels for diverse learners
- Automatic creation of practice problems and quizzes

2. Research Assistance:

- Literature summarization and synthesis
- Hypothesis generation and experimental design suggestions
- Draft paper writing and review support

Creative Industries

1. Media and Entertainment:

- Script writing assistance and character development
- Interactive storytelling with dynamically generated narratives
- Song lyrics and poetry generation

2. Visual and Multimodal Generation:

- Text-to-image generation (DALL-E, Midjourney, Stable Diffusion)
- Visual content editing and manipulation
- Cross-modal generation (text-to-video, text-to-3D)

Business and Enterprise

1. Code Generation and Software Development:

- Automated programming with tools like GitHub Copilot
- Bug detection and fixing
- Code documentation and explanation

2. Business Intelligence:

- Report generation from data
- Market analysis and trend identification
- Competitive intelligence summaries

4.1 Web Scraping for Data Collection, Techniques for Extracting Text from Websites, Meta Search, web spamming, Legal and Ethical Considerations

Web Scraping for Data Collection

Collecting document data from the Web is a significant task. The Web is a vast repository of documents, but these resources are widely distributed. To effectively use these resources for applications like search and knowledge discovery, it is necessary to download them at a central location.

This process of gathering Web pages is typically achieved through the use of Web crawlers, also known as spiders or robots. Web crawlers are automated programs designed to access Web pages non-interactively, unlike a user with a browser.

The basic crawler algorithm involves starting with a seed set of URLs added to a "frontier list". In each iteration, the crawler selects a URL from the frontier list using a selection algorithm, fetches the page using the HTTP protocol (the same mechanism browsers use), stores it in a local repository, extracts the URLs found within the page, and adds these new URLs to the frontier list if they have not been visited before. A separate data structure, often a hash table, is maintained to track visited URLs. The process continues until the frontier list is empty, although this doesn't mean the entire Web is crawled as the Web is not strongly connected.

Web crawlers can be categorised as:

Universal crawlers: These are used by commercial search engines like Google and Bing to periodically refresh downloaded Web resources, aiming to crawl pages irrespective of subject or location.

Preferential crawlers: These discriminate between the relevance of different pages for a specific application, such as crawling sites related to a particular subject for business intelligence.

The selection algorithm used by a crawler can introduce a bias, which can sometimes be desirable, for example, a breadth-first crawler is more likely to crawl a page with many links pointing to it, and this indegree is related to PageRank.

Techniques for Extracting Text from Websites

Once Web documents are collected, the first step for analysis is often to convert the raw text into a multidimensional format. This involves several preprocessing steps. For Web documents, specialised preprocessing techniques are required due to their structure and links.

Key techniques for extracting usable text from Web pages, as described in the sources, include:

Removing specific parts of the documents: This includes removing elements like HTML tags, which are generally not useful for mining applications.

Leveraging document structure: Different fields within HTML documents, such as the title, metadata, and body, are often treated with different levels of importance and weighted accordingly by analytical algorithms. The title of a document might be considered more important than the body and weighted more heavily.

Handling Anchor Text: Anchor text, which describes the page a link points to, is considered important due to its descriptive nature. However, it is sometimes not relevant to the topic of the page it appears on and is often removed from the text of that document. In some cases, anchor text can be added to the text of the document it points to, as it often serves as a summary description of that page.

Extracting text from the main content block: Web pages often contain irrelevant blocks such as advertisements, disclaimers, or notices. It has been shown that the quality of mining results improves when only the text from the main content block is used. The automated determination of the main block is itself a data mining problem, often relying on the similar layout used across documents on a specific site.

General Text Preprocessing: Beyond Web-specific steps, standard text preprocessing techniques are applied, such as stop-word removal (eliminating common, non-discriminative words like "a" or "the"), stemming (reducing words to their root form), and the removal of digits.

Parsing Web pages is also part of the process, for instance, to extract URLs.

Meta Search

Meta search is defined as the process of **retrieving search results from multiple search engines and combining them into a single list** for users. Its objective is to aggregate information from various sources to enhance the overall search experience.

Meta search engines use methods like similarity scores and rank positions to combine results from different search engines. They may also employ techniques aimed at mitigating bias.

Challenges and considerations for meta search include dealing with the diverse algorithms used by different search engines, handling query discrepancies where engines might interpret queries differently, and ensuring the timeliness and freshness of data from each source. Examples mentioned include Dogpile and Ixquick (now Startpage).

Web Spamming

Web spamming refers to unethical practices used to manipulate search engine rankings and deceive users. These practices often have economic incentives, such as a business owner wanting their site to rank highly for specific keywords.

The sources describe several strategies used for Web spamming:

Content Spamming: This involves filling a hosted Web page with repeated keywords, even if they are not visible to the user (e.g., by controlling text and background colour). This aims to

maximise content relevance for the search engine without increasing the visible relevance. Keyword stuffing, hidden text, and hidden links are forms of content spamming.

Cloaking: A more sophisticated approach where a Web site serves different content to crawlers than it does to users. The site determines if the request is from a crawler or a user; serving content relevant to keywords for the crawler and actual (e.g., advertising) content for the user. This misleads search engines about the page's true content.

Link Spamming: This involves artificially inflating link popularity. Methods include creating Link Farms (networks of sites solely for link exchange), Buying and Selling Links (purchasing links primarily to influence rankings), and Comment Spamming (posting irrelevant comments with links on blogs/forums). These practices violate search engine guidelines.

Hiding Techniques: These aim to mislead search engines or manipulate user experience. Examples include **IP Cloaking** (showing different content based on IP address, like to crawlers but not users), **JavaScript Redirects** (redirecting users differently from what search engines see), and **Doorway Pages** (pages optimised for keywords, often redirecting users, to manipulate rankings).

Search engines combat Web spam through algorithmic solutions that identify and penalise spam content and link schemes. Machine learning models are used to recognise patterns associated with spam.

Legal and Ethical Considerations for Web Scraping

Legal Issues:

- Terms of Service (ToS): Violating a website's ToS (e.g., prohibitions on scraping) may lead to lawsuits (e.g., hiQ Labs v. LinkedIn).
- Copyright Law: Scraped content (text, images) may be copyrighted; redistributing it without permission risks infringement.
- Data Privacy Laws: GDPR (EU), CCPA (California), and others regulate scraping personal data (e.g., emails, user profiles).
- Computer Fraud and Abuse Act (CFAA): Unauthorized access to websites (e.g., bypassing login walls) may violate U.S. law.

Ethical Concerns:

- Server Overload: Aggressive scraping can crash sites or degrade performance for others.
- Respect for Robots.txt: Ignoring this file (which specifies scraping rules) is unethical.
- Data Usage: Avoid scraping sensitive data (e.g., medical records) or using data for harmful purposes.

4.2 Sentiment Analysis and Opinion Mining: Analyzing Public Sentiments on Social Media, Case Studies in Opinion Mining

Sentiment Analysis and Opinion Mining

Sentiment analysis or opinion mining is defined as the computational study of people's opinions, appraisals, attitudes, and emotions towards entities, individuals, issues, events, topics, and their attributes. It is a computationally challenging and practically very useful task. The goal is to analyse opinions, sentiments, and emotions expressed in text.

The Problem and Need

With the rapid growth of social media on the Web (including reviews, forum discussions, blogs, and social networks), individuals and organisations are increasingly using public opinions from these media for decision-making. Businesses, for instance, constantly seek public or consumer opinions about their products and services. Potential customers also desire to know the opinions of existing users before making a purchase or using a service.

However, finding and monitoring these opinion sites and distilling the information they contain remains a significant task due to the proliferation of diverse sites, each containing a vast volume of opinionated text that is not always easily deciphered in long forum postings and blogs. Human readers struggle to identify relevant sites and accurately summarise the information and opinions. Moreover, human analysis is susceptible to considerable biases and mental/physical limitations when processing large amounts of information. Automated opinion mining and summarisation systems are thus needed to overcome these subjective biases and limitations with an objective sentiment analysis system.

An abstract model of sentiment analysis defines the problem and provides a common framework to unify different research directions. This abstraction often consists of two parts: opinion definition and opinion summarisation.

Distinct Aspects of Text in Social Media for Analysis

Textual data in social media presents many new challenges for traditional text analytics methods due to its distinct characteristics. These include:

Time Sensitivity: The relevance and meaning of text can change rapidly over time.

Short Length: Posts in social media are often very short.

Unstructured Phrases: The language used is often informal and contains non-standard vocabulary and grammatical errors.

Abundant Information: There is a huge volume of diverse user-generated content.

These features lead to problems like data sparseness and the semantic gap when applying traditional methods directly. Integrating semantic knowledge can be an effective way to bridge

this semantic gap. Social media mining specifically addresses the challenge of mining user-generated content with social relations.

Levels and Techniques of Sentiment Analysis

Opinion mining can be approached at different levels and using various techniques:

1. **Document Sentiment Classification:** This involves classifying an opinion document, such as a product review, as expressing a positive or negative opinion or sentiment. This is also known as document-level sentiment classification.

Supervised Learning: This is often formulated as a supervised learning problem (e.g., positive, negative, neutral classes). Existing methods like naive Bayesian classification and Support Vector Machines (SVM) can be applied. Product reviews with reviewer-assigned ratings (e.g., stars) are commonly used as training data. Features used can include unigrams (bag of words) and other engineered features.

Unsupervised Learning: This approach often relies on known opinion words and phrases. One method extracts phrases containing adjectives or adverbs along with a context word, as the opinion orientation of a word can depend heavily on context.

2. **Sentence Subjectivity and Sentiment Classification:** This involves two sub-tasks for a given sentence:

Subjectivity classification: Determining if a sentence is subjective (expressing personal feelings/views) or objective (presenting factual information).

Sentence-level sentiment classification: If a sentence is subjective, determining if it expresses a positive, negative, or neutral opinion.

These tasks can filter out sentences with no opinions and help determine opinion polarity about entities and aspects within the sentence. Traditional supervised learning methods are applicable. Some methods use seed words and score functions.

A common assumption is that a sentence expresses a single opinion from one holder, but complex sentences can have mixed opinions. It is important to note that many objective sentences can imply opinions too, which are difficult to recognise as they often require domain-specific common-sense or world knowledge. Mining opinions requires considering both subjective and objective sentences.

3. **Aspect-Based Sentiment Analysis:** While document and sentence-level analysis are useful, they lack the detail needed for many applications. A document might have an overall positive sentiment but contain negative opinions on specific aspects of the entity. Aspect-based sentiment analysis aims to determine opinions at the level of specific aspects or features of an entity.

This requires deeper natural language processing capabilities.

Methods can include applying sentence-level classification to sentences or clauses containing aspects.

Lexicon-based approaches have been shown to perform well and can handle mixed opinions or opinions needing phrase-level analysis, although they assume entities and aspects are known beforehand.

Aspect extraction (finding the entities/aspects opinions are about) is a key problem. Unsupervised methods for extracting aspect expressions, often nouns and noun phrases, have been developed.

Opinion lexicon expansion (generating lists of opinion words and phrases) is instrumental. This can be done via dictionary-based or corpus-based approaches, considering sentiment consistency.

Techniques can also use dependency grammar to model relationships between opinion words and aspects. Double-propagation methods can extract both opinion words and aspects together.

4. Using Topic Models: Topic modeling, which performs dimensionality reduction on unstructured text by learning low-dimensional thematic representations (topics), can be applied to sentiment analysis.

Topic Sentiment Mixture (TSM) models weblogs as a mixture of background, topic, and positive/negative sentiment language models, simultaneously detecting topics and sentiments.

Supervised Topic Modeling (sLDA) incorporates response variables or covariates (like user demographics or political leanings) to improve topic quality and learn predictive models. This allows analysis of what topics different user groups discuss.

Relational Topic Modeling (RTM) incorporates relational information like social graphs or links to assist topic modeling.

5. Cross-Lingual Opinion Mining: This involves performing opinion mining on text in one language using resources (like annotated corpora or classifiers) from other languages. Machine translation is typically used. This is useful when annotated resources for the target language are limited or when multi-language comparable results are needed.

Opinion Spam Detection

Opinion spam refers to human activities that deliberately mislead readers or automated systems by giving undeserving positive or negative opinions, often for financial gain. These are also called fake or bogus opinions. Detecting opinion spam is crucial as people increasingly rely on online opinions for decision-making.

Methods for detecting opinion spam can be based on supervised learning, abnormal behaviors, or group spam detection.

Spam opinions add noise to the mining process and are not useful.

Case Studies and Applications

Customer Reviews: Mining customer reviews or opinions, often short text documents, to determine useful information. Product reviews (e.g., digital cameras, mattresses) are frequently used as data sources. The goal is to understand customer sentiments.

Social Media Analysis: Analysing text data in social media, such as product reviews or opinions, to reveal and summarise opinions about a topic for applications like supporting consumer decisions and business intelligence. This includes text from blogs, microblogs, discussion forums, and social networks.

Public Opinion and Polling: Using text sentiment from sources like tweets to link to public opinion time series. Studies have shown that carefully selected social media data can be used as a lens to analyse and even predict real-world events, such as correlating Twitter data with Gallup opinion polls for presidential job approval.

Business Intelligence: Exploiting opinionated text data for business intelligence purposes.

4.3 Opinion Spam Detection: Information Retrieval and Search Engines: Introduction to Search Algorithms, Building a Basic Search Engine Social Network Analysis: Analyzing Social Media Networks, Identifying Influencers and Trends

Sentiment Analysis and Opinion Mining

Sentiment analysis, also known as opinion mining, is the computational study of people's opinions, appraisals, attitudes, and emotions towards entities, individuals, issues, events, topics, and their attributes. It is described as a computationally challenging yet practically useful task. The goal is to analyse opinions, sentiments, and emotions expressed in text.

The proliferation of social media on the Web, including reviews, forum discussions, blogs, and social networks, has led to increased use of public opinions from these sources for decision-making by individuals and organisations. Businesses and potential customers alike seek public or consumer opinions about products and services. However, the volume of opinionated text across diverse sites makes finding, monitoring, and distilling this information a significant task. Automated opinion mining and summarisation systems are needed to overcome subjective biases and limitations of human analysis when processing large amounts of information.

An abstract model of sentiment analysis typically consists of opinion definition and opinion summarisation. Opinion mining can be applied at different levels, including document sentiment classification, sentence subjectivity and sentiment classification, and aspect-based sentiment analysis. Various techniques are used, such as supervised and unsupervised learning for

classification and lexicon-based approaches for aspect-based analysis. Topic modeling, like Latent Dirichlet Allocation (LDA) and its variations like supervised LDA (sLDA) and Relational Topic Modeling (RTM), can also be applied to sentiment analysis. Opinion mining from text data to reveal and summarise opinions about a topic has widespread applications, such as supporting consumer decisions and business intelligence.

Opinion Spam Detection

A significant issue in opinion mining is **opinion spam**, which refers to human activities that deliberately mislead readers or automated systems by providing undeserving positive or negative opinions. These are also called fake or bogus opinions. Opinion spam often occurs for financial gain. Detecting opinion spam is crucial because people increasingly rely on online opinions for their decision-making. Spam opinions add noise to the mining process and are not useful.

Methods for detecting opinion spam can be based on supervised learning, abnormal behaviors, or group spam detection. Group spam detection, for instance, might involve discovering groups of reviewers and then ranking them based on indicators of unusual group behaviours, such as writing reviews together in a short time window, writing reviews right after a product launch, group content similarity, or group rating deviation. Supervised learning can be used in spam detection. The problem of detecting fake or spam opinions was introduced by Jindal and Liu.

Text Analytics and Social Media

Social media is one of the most common sources of text on the web. Textual data in social media presents new challenges for traditional text analytics due to its distinct characteristics: time sensitivity, short length, unstructured phrases, and abundant information. These features can lead to problems like data sparseness and the semantic gap. Social media mining specifically addresses the challenge of mining user-generated content with social relations. Opinion mining from text data is a key application in social media. Text analytics techniques are used to efficiently deal with textual data in social media for research and business purposes.

Applying text analytics to social media includes tasks such as Event Detection, Collaborative Question Answering, and Social Tagging. Mining text in social media requires the ability to handle dynamic data with poor and non-standard vocabulary. The text may occur within linked social networks, and these links can be used to improve mining quality.

Information Retrieval and Search Engines

Search engines are typically used to manage text data due to the lack of structure. They allow users to find useful information from a collection with a keyword query. Improving the effectiveness and efficiency of search engines has been a central research topic in information retrieval.

Search engines generally involve components like crawling (gathering web pages), indexing (creating an inverted index for fast retrieval), and ranking (determining relevance to a query). Search algorithms are crucial for ranking pages. Two common algorithms used for ranking in search engines are **PageRank** and **HITS**.

- **PageRank** models the importance of Web pages using the citation (linkage) structure. The basic idea is that reputable documents are likely linked by other reputable pages. It models the reputation as the long-term frequency of visits by a random surfer (steady-state probability). PageRank can be considered a rank centrality score in networks. It can be used to generate features useful for ranking in social systems.
- **HITS** (Hyperlink-Induced Topic Search) is another algorithm for computing influential nodes in a webgraph. Unlike PageRank, it considers two types of nodes: authority nodes (pages with many in-links) and hub nodes (pages with many out-links). Authorities have in-links from many hubs. HITS employs an iterative approach to compute authority and hub scores.

Search engines deploy algorithmic solutions, often including machine learning models, to identify and penalise spam content and link schemes (Web Spamming). Web spamming refers to unethical practices to manipulate search engine rankings. This concern parallels the need for opinion spam detection in opinion mining. User feedback or behavioural analysis mechanisms can provide evidence of relevance in search results and are important for commercial recommender systems, where recommendations are tailored based on user preferences.

Social Network Analysis: Analysing Social Media Networks, Identifying Influencers and Trends

Social network analysis is an integral part of social media mining. It involves studying how individuals interact and how communities form in social media. Methods from this field are used to formally represent, measure, model, and mine meaningful patterns from large-scale social media data.

Social network analysis can be used for finding related communities of users (community detection), anomaly detection, classification, influence analysis, and link prediction. Measures like degree centrality, eigenvector centrality (related to PageRank), Katz centrality, PageRank centrality, betweenness centrality, and closeness centrality are used to find the most central or influential nodes in a graph. The Iterative Classification Algorithm (ICA) can use content associated with nodes, such as user posts in social networks, for classification, integrating both link and content features.

Within social media, researchers utilise various information like posts, links, and tags to identify influential users in platforms like the blogosphere and microblogosphere. Understanding user behaviour in microblogosphere by analysing user intentions at a community level is also studied. Social Influence Analysis is an active topic in social media mining.

Event detection in social media is another application, where the frequency of tweeting, for example, can be used to identify events. Real-time event detection by social sensors has been explored using tweets.

Connecting Opinion Spam Detection, Information Retrieval, and Social Network Analysis

Opinion spam detection, a critical task within opinion mining, directly relates to both Information Retrieval and Social Network Analysis when applied to social media data.

1. **Information Retrieval Link:** Opinionated text resides on the web and social media platforms. Finding this text, whether for aggregation, summarisation, or analysis, relies on techniques from Information Retrieval (IR). Search engines provide the tools to access this vast content. Just as IR faces the challenge of web spam, opinion mining faces the challenge of opinion spam. The goal of both is to ensure that users find trustworthy and relevant information. While standard search algorithms like PageRank and HITS focus on ranking web pages based on link structure and content relevance, techniques for combating general web spam share the underlying motivation of filtering misleading content, similar to opinion spam detection. Furthermore, IR methods are used to find opinions, and opinion spam detection adds a layer of filtering to improve the quality of the retrieved opinions.
2. **Social Network Analysis Link:** Social media is the primary source for much opinion mining and spam detection work. Opinionated text is often embedded within a social network structure, where users are connected. Analysing this network structure and user behaviour (Social Network Analysis) provides crucial context for detecting opinion spam. For example, identifying groups of users acting unusually within the network (group spam detection) is a Social Network Analysis task applied to opinion mining. Features related to authors' identities and social networks can improve the prediction of review quality, which is related to identifying non-spam, useful opinions. Identifying influential users might also be relevant, as spammers could potentially manipulate these measures to appear more credible, or influencers might be targeted by spammers. Understanding user behaviour, a core aspect of social media mining and social network analysis, can help in identifying patterns indicative of spamming activity.

In essence, opinion spam detection is a specialized form of content filtering (related to IR's spam combating) applied to opinionated text (the subject of sentiment analysis/opinion mining), often leveraging the social context and behavioural patterns available in social media platforms (the domain of Social Network Analysis). All three fields contribute techniques and understanding necessary for effectively analysing and filtering online opinions to combat misleading information.